

System and Devices (SYS3)

Lecture Review – Week 9

Dr Pengcheng Liu

Welcome



Check-In Code:

280707

Log into checkin.york.ac.uk, find the session and enter the code.

Valid until Thursday, 23 April 2026 11:00

Week 9 Module Overview and Updates

Module Overview and Updates



- **Week 9:**

Lecture: In Week 9, our discussion was focused on some principles of cache memory, and block replacement & write strategy in Lecture 15 and Lecture 16, respectively.

Lab: Instruction Reordering. Instruction reordering can be used for a number of reasons, to reduce hazards, including structural hazards, and data hazards. In this exercise we focus on using reordering mainly as a way to improve performance where register hazards persist. For example, we know from previous exercises that RAW hazards cannot be eliminated by renaming, although it can eliminate WAR and WAW hazards. Reordering the instruction sequence can sometimes help to improve if not eliminate these problems. This session will investigate reordering and look at ideas for solving this problem - though the full solution for reordering is advanced and beyond the scope of this module exercise sessions.

Module Overview and Updates



- **Week 9:**

Exercises/Quizzes:

Quiz 3 will be discussed in this Lecture Review session.

Exercise 3 solution will be released on 24/04/2026.

Exercise 3 will be discussed in the Lecture Review session on 30/04/2026.

Module Overview and Updates



- **Next Week (Week 10):**

Lecture: In Week 10, our discussion will be focused on some cache memory optimization techniques in Lecture 17 and the 4th Tutorial Lecture on Caches.

Lab: In Week 10 there will be no algorithm coding. However we will use the pipeline toolset to explore some tradeoffs relating to power consumption of the CPU and the effect of optimisation.

Exercises/Quizzes:

Exercise 3 will be discussed in the Lecture Review session on 30/04/2026.

Questions on [Padlet](#)

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2: Processor Forwarding

When you talk about a processor with "forwarding" (as you do in the Sample Paper), how do we know whether this refers to Operand Forwarding as in the 5 stage pipeline where the output of the alu is fed back in, or as in Tomasulo-style CDB Forwarding?

Recap:

Two Kinds of Forwarding — Same Goal, Different Worlds

Operand Forwarding in the Classic 5-Stage Pipeline

- Also called **data forwarding** or **bypass forwarding**.
- Happens in **in-order execution pipelines** (like MIPS 5-stage).
- If an instruction needs a value that's **being produced by an earlier instruction** but hasn't yet been written back to the register file, the processor forwards the result **directly from the ALU output** (EX stage) to a future instruction's input.
- This avoids stalls due to **read-after-write (RAW) hazards**.

An ALU result is passed **backward** from the EX stage to the input of the next instruction in ID or EX.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2: Processor Forwarding

When you talk about a processor with "forwarding" (as you do in the Sample Paper), how do we know whether this refers to Operand Forwarding as in the 5 stage pipeline where the output of the alu is fed back in, or as in Tomasulo-style CDB Forwarding?

Tomasulo-Style Forwarding (CDB Forwarding)

- Used in **out-of-order execution architectures** (like Tomasulo's algorithm).
- Instructions sit in **reservation stations**, waiting for operands.
- When a functional unit completes execution, the result is **broadcast on the Common Data Bus (CDB)**.
- Any reservation station (or register) **listening** for that value can grab it immediately — that's **forwarding via the CDB**.

The result of an instruction is **broadcast to many potential consumers** across the chip, not just the next instruction in order.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2: Processor Forwarding

When you talk about a processor with "forwarding" (as you do in the Sample Paper), how do we know whether this refers to Operand Forwarding as in the 5 stage pipeline where the output of the alu is fed back in, or as in Tomasulo-style CDB Forwarding?

How to distinguish? - Look at the context:

- If we're discussing a **classic 5-stage pipeline** (as in the sample paper), **it refers to operand forwarding within the pipeline stages.**
- If the topic is **dynamic scheduling, Tomasulo's algorithm, or out-of-order execution**, then **forwarding refers to broadcasting results via the CDB to multiple dependent instructions.**

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2

RSIs

Are RSIs a Tomasulo's Algorithm specific thing? Or do they have wider application?

RSIs are a key part of Tomasulo's Algorithm.

The **mechanism** of RSIs appears in many out-of-order CPU designs, often in more advanced or renamed forms. For example, Register Alias Table (RAT) in Modern CPUs.

The core idea is to track where the latest value will come from.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2

Reservation Station : Functional Unit

What is the relational ratio between Reservation Stations and Functional Units? Are there multiple reservation stations per functional unit? Or does each reservation station have its own functional unit? Slide 24 would suggest that each functional unit has exactly one reservation station, but the topology map suggests otherwise?

Yes, there are typically multiple reservation stations per functional unit. The 1:1 mapping you saw in the lecture is a simplification used for clarity.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2

RS Fields and Loads

S21 says that Vk holds the offset (immediate value) for a load, but S22 says that A holds this value. Which is it? Or is it both? Or am I missing part of the puzzle? And possibly relatedly, why does the example scenario from S24 onwards not store the immediate value of the load instruction in Vk?

In the context of the Tomasulo's algorithm implementation described in Slides 21 and 22, both Vk and A can be involved in the handling of the offset for a load instruction, but they serve slightly different purposes at different stages of execution.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2

RS Fields and Loads

During **Instruction Issue**:

For a load instruction like **Load F6, 32(R2)**, we have:

- R2 as the base register
- 32 as the offset (immediate)

The Vk field is temporarily used to hold the offset value (32 in this case).
In this case,

- Load instructions only have one real source register (R2), so Qj and Vj are used to track the base register value.
- Since the second operand (the offset) is not a register but an immediate constant, it doesn't need dependency tracking — and Vk is available to simply hold that constant.

Padlet Questions

S2 Week 6 L10 - Tomasulo's algorithm 2

RS Fields and Loads

During **Execution (Address Calculation)**:

The A field is used to hold the address information:

- Initially, it may store the immediate offset.
- But after address computation ($R2 + 32$), it holds the effective address to be used for memory access.

In effect:

- Vk holds the offset early, when operands are being gathered.
- A holds the result of base + offset, just before the memory access.

In practice, implementations may skip using Vk entirely for loads and go straight to populating A with either:

- Just the offset, and then add the base register during execution, or
- the full effective address, if the base register value is already available.

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Please can you go over the relationship between addresses of something in memory and the say this is translated to being found in a cache?

Let's assume the following cache setup for our example:

- **Cache size:** 1KB (1024 bytes)
- **Block size:** 16 bytes
- **Number of cache blocks (lines):** $1024 / 16 = 64$

So, we have **64 cache lines**, numbered 0 to 63

This means we need **6 bits** (because $2^6 = 64$) to identify which cache line we're talking about.

Let's assume a **32-bit CPU address** space.

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Step 1: CPU Requests a Memory Address

Let's say the CPU wants to access memory address:

0x12345678

The CPU is requesting a specific word (or byte, depending on instruction), not a whole block. But caches are block-based, so the system has to fetch the whole block that contains that word.

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Step 2: Break Down the Address

Every memory address can be divided into 3 parts when used in cache:

- 1. Offset – how far into the block the data is

Block size is 16 bytes → we need 4 bits to know where we are inside the block ($2^4 = 16$)

- 2. Index – which cache line to look in

64 cache lines → we need 6 bits to choose one

- 3. Tag – the remaining upper bits that uniquely identify the memory location

Let's break down address 0x12345678 (in binary):

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Step 2: Break Down the Address

Let's break down address 0x12345678 (in binary):

```
0x12345678 = 0001 0010 0011 0100 0101 0110 0111 1000
                ↑           ↑           ↑           ↑
                Tag       Index   Offset   (in bits)
```

We divide it up like this:

- Offset (last 4 bits): bits [3:0] → 1000 (that's 8 in decimal)
- Index (next 6 bits): bits [9:4] → 011001 (that's 25 in decimal)
- Tag (remaining bits): bits [31:10] → 0001 0010 0011 0100 0101 01 (22 bits)

So:

- Offset = 8 → byte 8 inside the block
- Index = 25 → check cache line 25
- Tag = a 22-bit value → this tells us which block of memory is in that cache line

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Step 3: Look in the Cache

Go to cache line 25 (because index = 25)

That line stores:

- A tag (saved from a previous memory load)
- A valid bit (is the data valid?)
- The 16 bytes of data for that block

Now:

- Check if the tag stored in line 25 matches the tag from our address (0x12345678)
- If the tags match and the valid bit is 1 → cache hit
- If the tags don't match or the valid bit is 0 → cache miss

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

If It's a Hit

- Use the offset (8) to get the exact byte/word from the block in cache line 25

If It's a Miss

- The cache must load the entire 16-byte block from memory that includes address 0x12345678
- Load this block into cache line 25
- Store the tag (from address) in the tag field for line 25
- Set valid bit = 1

Then go back to extract byte 8 from the block.

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Short answer to the questions:

1. Does the CPU request a block? The CPU requests a specific address (a byte or word). The cache handles blocks, so the system has to map that address into a block behind the scenes.
2. Is the Index part of the original address? Yes, indirectly. The index bits come from the original address. We divide the address into offset, index, and tag based on the cache configuration.
3. What is the Tag? The Tag is the upper part of the address that uniquely identifies a block of memory. It's used to make sure the block in the cache actually matches the requested address.
4. Where does the Offset come from? The Offset comes from the lowest bits of the address and tells us the position inside a block. If the block is 16 bytes, we need 4 bits of offset to specify one of those 16 bytes.

Padlet Questions

S2 Week 9 L15 - Cache intro 1

Addresses

Visual summary

If cache has:

- Block size = 2^b
- Number of lines = 2^n

Then

```
Address (e.g., 32-bit) = [ TAG | INDEX | OFFSET ]  
                      = [ 32 - n - b bits | n bits | b bits ]
```

Each address is split like this to find:

- Which cache line to look in (INDEX)
- Whether it's the right data (compare TAG)
- Where inside the block to find it (OFFSET)

Padlet Questions

S2 Week 9 L15 - Cache intro 1 CPU-Cache Interaction

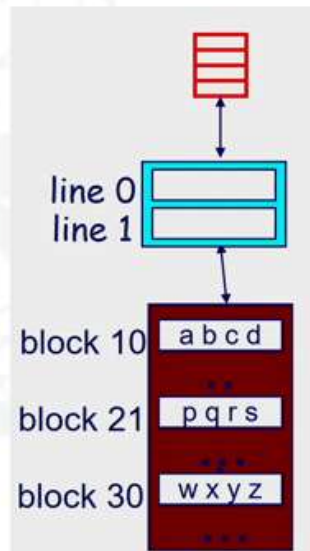
Presumably this is only an example and in some systems the units of transfer will be different?

Yes this is only an example to visualize the roles of registers, cache and memory, and in real-world systems, the units of transfer can be varying depending on the architecture, cache design, and performance goals.

CPU-Cache Interaction



- The transfer unit between the CPU register file and the cache is a 4-byte word
- The transfer unit between the cache and main memory is a 4-word block (16B)



- The tiny, very fast CPU register file has room for four 4-byte words
- The small fast L1 cache has room for two 4-word blocks
- The big slow main memory has room for many 4-word blocks

Padlet Questions

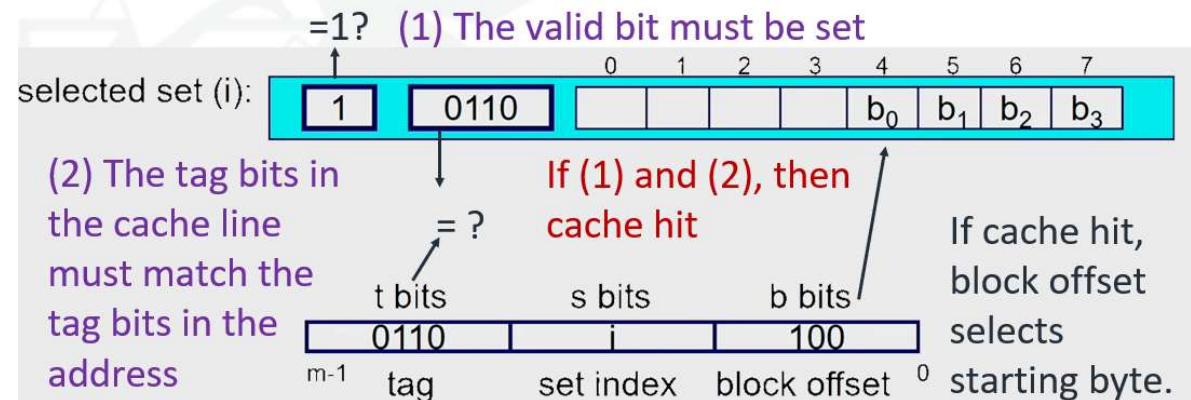
S2 Week 9 L15 - Cache Introduction – Accessing Direct-Mapped Caches Slide 21:

Can you explain this again, I don't understand the whole offset thing.

Once it's determined that there is a **cache hit** (by meeting the two conditions: the valid bit must be set and the tag bits in the cache line must match the tag bits in the address), the desired data is present in the cache. Then the cache controller retrieves the cache line corresponding to the index where the data is stored.

Accessing Direct-Mapped Caches

- Block matching: Find a valid block in the selected set with a matching tag
- Word selection: Then extract the word



Within this cache line, the **block offset** is used to select the byte within the cache block where the desired data resides. Since the cache block size is fixed, the block offset specifies the position of the desired byte relative to the start of the cache block.

Padlet Questions

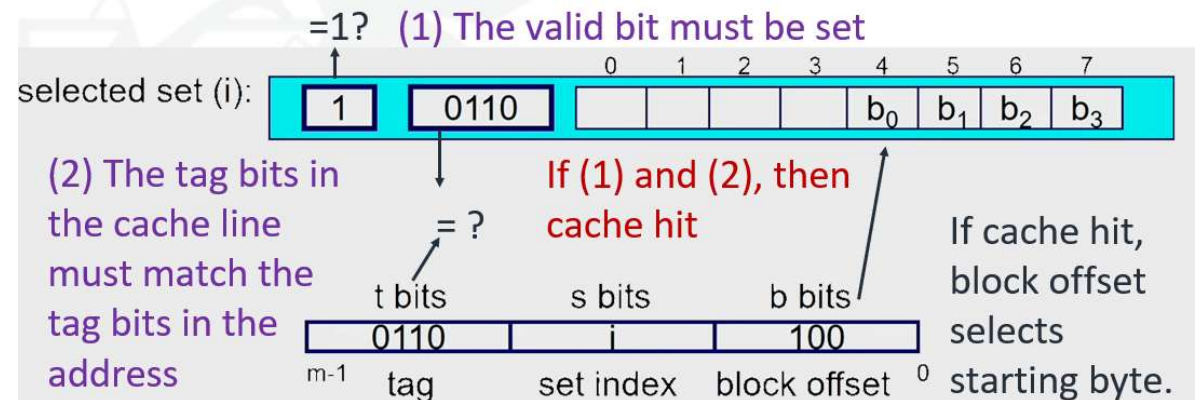
S2 Week 9 L15 - Cache Introduction – Accessing Direct-Mapped Caches Slide 21:

Can you explain this again, I don't understand the whole offset thing.

Accessing Direct-Mapped Caches

For example, if the block offset is 6 bits, it can represent values from 0 to 63 (since $2^6 = 64$), covering all bytes within the 64-byte cache block. The specific byte within the cache block to be accessed is determined by the value of the block offset.

- Block matching: Find a valid block in the selected set with a matching tag
- Word selection: Then extract the word



The block offset determines the starting byte within the cache block by specifying the position of the desired byte relative to the beginning of the cache block.

Padlet Questions

S2 Week 9 L15 - Cache Introduction – Accessing Direct-Mapped Caches Slide 21:

Can you explain this again, I don't understand the whole offset thing.

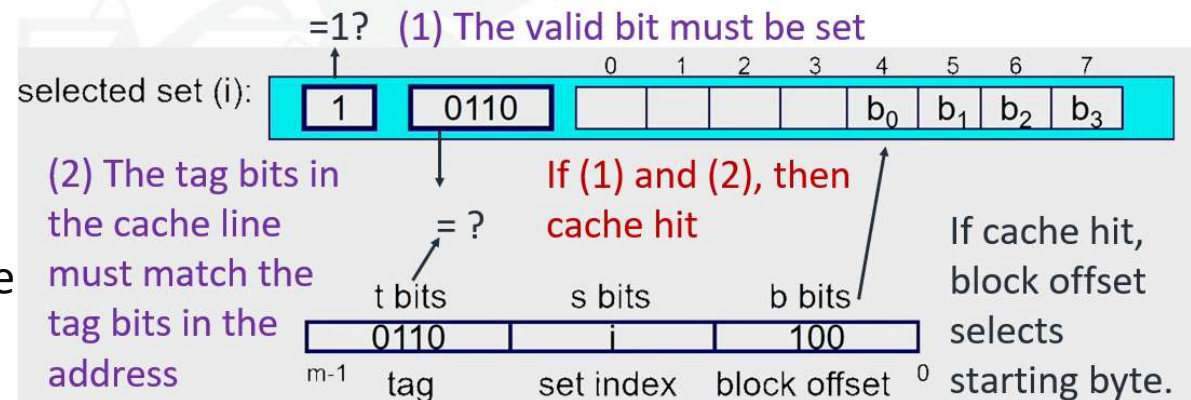
Each value of the block offset corresponds to a specific byte position within the cache block.

For example:

- If the block offset is 0, it refers to the first byte within the cache block.
- If the block offset is 1, it refers to the second byte within the cache block.
- If the block offset is 63, it refers to the last byte within the cache block.

Accessing Direct-Mapped Caches

- Block matching: Find a valid block in the selected set with a matching tag
- Word selection: Then extract the word



To select the starting byte within the cache block, the cache controller combines the block offset value with the starting address of the cache block. This calculation determines the memory location of the desired byte within the cache block.

Once the starting byte is determined, the cache controller can efficiently access the desired data from the cache block.

Padlet Questions

S2 Week 9 L15 - Cache Introduction - Cache Indexing Decoder: What is the cache indexing decoder actually used for? I'm a bit confused about WHY the index needs decoding?

As discussed in lecture, **the index is used to determine the location of a particular memory block in the cache.** It specifies which set of memory blocks the block belongs to.

However, the index itself is often represented in binary format, which may not be easily interpreted by the cache controller. So a cache indexing decoder is used to **interpret the binary index and determine the physical location** of the memory block in the cache. The decoder takes the binary index as input and produces a set of control signals that are used to select the appropriate memory block from the cache.

Cache Indexing

- Decoders are used for indexing
- Indexing time depends on decoder size ($s: 2^s$)
- Smaller number of sets, less indexing time

t bits		s bits	b bits
		0 0 0 1	
m-1	tag	set index	block offset0

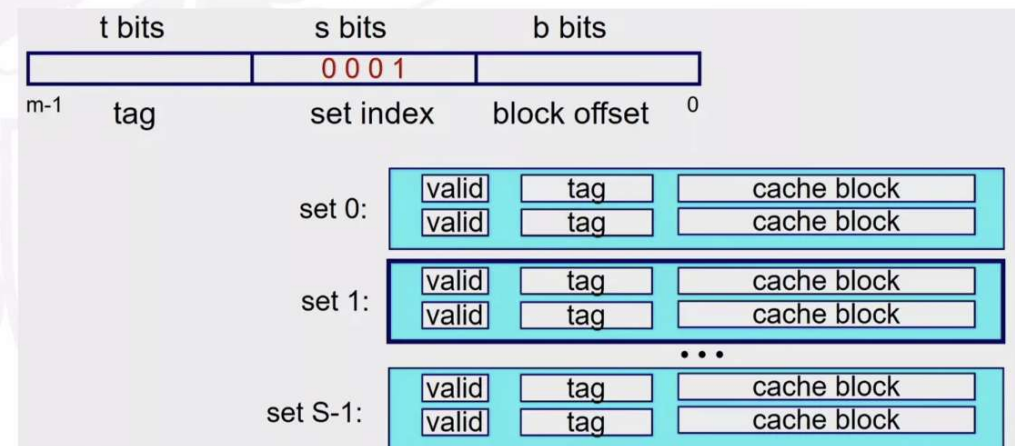
Padlet Questions

S2 Week 9 L15 - Cache Introduction - Tag Matching: When comparing the tags to match them, is the given tag to match (with the offset etc.) given by the CPU?

The tag to be compared with the address of a memory block is typically **generated by the CPU**. When the CPU initiates a memory access, it sends the memory address along with the command to read or write to the cache controller.

Accessing Set Associative Caches

- Set selection is identical to direct-mapped cache



As discussed in lecture, the memory address typically includes several components, including the tag, the index, and the offset. The tag is used to identify which memory block is being accessed, the index is used to determine which set of memory blocks the block belongs to, and the offset is used to specify the position of the data within the block.

Padlet Questions

S2 Week 9 L15 - Cache intro 1 Cache Size

I'm confused about this calculation: You say it that $C=B \times E \times S$, but each line in each set in the diagram has more than just B bytes; it also has 1 valid bit and t tag bits (which are shown as being separate to the B bytes in the diagram). Why are these not factored in to the calculation?

This formula gives you the data capacity of the cache: C = total size of data stored in the cache (in bytes); B = block size (number of bytes per block or cache line); E = number of lines (blocks) per set (i.e., the associativity); S = number of sets.

So, $B \times E \times S$ tells you the total amount of actual data the cache can store, excluding metadata like tags and valid bits.

The formula is specifically for the cache's data storage capacity, not the total memory footprint of the cache hardware. The valid bits and tag bits are considered metadata, not part of the actual data stored. They are required for cache management, but they don't contribute to the size of the data the cache can store and return to the processor.

Padlet Questions

S2 Week 9 L16 - Block Replacement- Dirty and Clean Blocks: I was watching this lecture and you mention about maintaining whether a block is clean or dirty. Can you confirm what this means? Thanks

A clean block refers to a block that has **not been modified** since it was brought into the cache. In other words, the data in the block in the cache is identical to the data in the main memory or storage device. This means that the block can simply be discarded without needing to be written back to the main memory or storage device, which can save time and reduce data transfer overhead. A dirty block, refers to a block that **has been modified** since it was brought into the cache. In other words, the data in the block in the cache is different from the data in the main memory or storage device. If a dirty block is replaced without being written back to the main memory or storage device, the changes made to the block will be lost. Therefore, before a dirty block is replaced, it must be written back to the main memory or storage device to ensure that the changes are saved. The distinction between clean and dirty blocks is important because it affects how the algorithm chooses which block to replace next.

Module Survey



UNIVERSITY
of York

Systems & Devices 3: Advanced Computer Systems



Survey details

Response rate progression

Participants	275
Response count	41
Response rate	14.91%
0% - 15% 15% - 30% 30% - 100%	
Last survey submitted	22 Apr 2026, 23:59:14
Survey open	20 Apr 2026, 07:00:00
Survey reminder to participants	27 Apr 2026, 09:00:00
Response rate notification	28 Apr 2026, 09:00:00
Survey close	5 May 2026, 09:00:00

Refresh ↻

Open surveys can be accessed by students at <https://york.surveys.evasysplus.co.uk/>

Scan QR code to access all open surveys





UNIVERSITY
of York

SYS3 Exercises/Quizzes Discussions

Quiz 3

Quiz 3



UNIVERSITY
of York

Question 1

1 point



_____ is used to implement virtual memory organisation.

- (A) Page table
- (B) Frame table
- (C) MMU
- (D) None of the mentioned

(A) (A)

(B) (B)

(C) (C)

Correct answer

(D) (D)

The MMU stands for Memory Management Unit and it is used to implement virtual memory organization.

Quiz 3



UNIVERSITY
of York

Question 2

1 point



_____ translates the logical address into a physical address.

- (A) MMU
- (B) Translator
- (C) Compiler
- (D) Linker

(A) (A)

Correct answer

(B) (B)

(C) (C)

(D) (D)

The MMU translates the logical address into a physical address by adding an offset.

Quiz 3



UNIVERSITY
of York

Question 3

1 point



The associatively mapped virtual memory makes use of _____

- (A) TLB
- (B) Page table
- (C) Frame table
- (D) None of the mentioned

(A) (A)

Correct answer

(B) (B)

(C) (C)

(D) (D)

Question 3

The associatively mapped virtual memory makes use of TLB.

Associatively mapped virtual memory refers to how virtual memory addresses are matched (or *mapped*) to physical memory addresses. In this type of mapping, any virtual address can be associated with any physical address — there's no fixed rule like in direct mapping. It's a *flexible* way to manage memory.

To make this efficient, especially since full associativity can be slow if you have to search a big table, we use a **TLB**.

Question 3

The associatively mapped virtual memory makes use of TLB.

TLB stands for **Translation Lookaside Buffer**.

It's a small, very fast cache that stores recent translations from **virtual addresses** to **physical addresses**.

So basically:

When a program generates a virtual address, the system first checks the TLB.

If the TLB has the translation (a TLB hit), you immediately get the physical address — fast!

If it's not in the TLB (a TLB miss), then the system has to look in the page table (which is slower), and then it usually updates the TLB with this new translation.

Quiz 3



UNIVERSITY
of York

Question 4

1 point



The effectiveness of the cache memory is based on the property of _____

- (A) Locality of reference
- (B) Memory localisation
- (C) Memory size
- (D) None of the mentioned

(A) (A)

Correct answer

(B) (B)

(C) (C)

(D) (D)

This means that the cache depends on the location in the memory that is referenced often.

Question 4

The effectiveness of the cache memory is based on the property of locality of reference.

What is Locality of Reference?

This is a behaviour pattern in programs where memory accesses are not random — they tend to be clustered in predictable ways. There are two main types:

Temporal Locality (Time-based):

If a program accesses some data (like a variable or instruction), it's **likely to access it again soon**.

Example: A loop counter used in every loop iteration.

Spatial Locality (Space-based):

If a program accesses one memory location, it's **likely to access nearby memory locations soon**.

Example: Accessing elements in an array, one after another.

Question 4

The effectiveness of the cache memory is based on the property of locality of reference.

How This Makes Cache Effective

Cache memory works best **because** programs tend to follow the locality of reference.

Temporal locality → Cache stores recently used data → If it's needed again soon, it's already in cache → Fast access!

Spatial locality → Cache loads not just the requested data, but nearby data too → So upcoming accesses are already in cache.

Quiz 3



UNIVERSITY
of York

Question 5

1 point



Let A1 represents an event of playing a video file by accessing the pixel contents of adjacent frames of the video from memory. And let A2 represents an event of initializing a 1D array with the value zero on all array elements sequentially using a for loop. Which of the following is TRUE?

- (A) Neither A1 nor A2 shows temporal or spatial locality.
- (B) A1 shows temporal locality and A2 shows spatial locality.
- (C) Both A1 and A2 show spatial locality.
- (D) A1 shows spatial locality and A2 shows temporal locality.

(A) (A)

(B) (B)

(C) (C)

Correct answer

(D) (D)

Quiz 3

Question 6

1 point



The performance of the system is greatly influenced by increasing the level 1 cache.

True

Correct answer

False

This is so because the L1 cache is onboard the processor.

Question 6

The performance of the system is greatly influenced by increasing the level 1 cache.

Quick recap: What is L1 Cache?

L1 Cache is the **smallest and fastest** cache in the memory hierarchy. It sits **closest to the CPU core** — even inside the processor. It's usually split into **instruction cache (I-Cache)** and **data cache (D-Cache)**.

Why is L1 Cache Important?

The CPU runs at *very high speed*, and it needs data **fast**. If the data it needs is not in L1 cache (a *miss*), it has to check L2, L3, or even RAM — which are **slower**.
L1 hits = **super fast** (nanoseconds or less).
L1 misses = **delay** (can be 5x, 10x, or more slower).

Question 6

The performance of the system is greatly influenced by increasing the level 1 cache.

Why does increasing L1 cache improve performance?

- **Faster Data Access**

L1 cache is faster than other levels of memory (like L2, L3, or RAM).

More cache = more data stored close to the CPU.

Result: **CPU spends less time waiting** for data → faster program execution.

- **Fewer Cache Misses**

A bigger L1 cache means a **higher chance** the needed data is already there.

Fewer cache misses = fewer slow trips to L2 cache or main memory.

This leads to **better CPU efficiency**.

- **Lower Latency**

Accessing L1 cache can take just a few CPU cycles (e.g., 1–4 cycles).

If the CPU can find more of its needed data in L1, it **maintains high speed** without stalling.

Quiz 3



UNIVERSITY
of York

Question 7

1 point



The main purpose of having memory hierarchy is to _____

- (A) Reduce access time
- (B) Provide large capacity
- (C) Reduce propagation time
- (D) Reduce access time & Provide large capacity

(A) (A)

(B) (B)

(C) (C)

(D) (D)

Correct answer

By using the memory Hierarchy, we can increase the performance of the system.

Quiz 3



UNIVERSITY
of York

Question 8

1 point



The spatial aspect of the locality of reference means _____

- (A) That the recently executed instruction is executed again next
- (B) That the recently executed won't be executed again
- (C) That the instruction executed will be executed at a later time
- (D) That the instruction in close proximity of the instruction executed will be executed in future

(A) (A)

(B) (B)

(C) (C)

(D) (D)

Correct answer

The spatial aspect of locality of reference tells that the nearby instruction is more likely to be executed in future.

Quiz 3



UNIVERSITY
of York

Question 9

1 point



The temporal aspect of the locality of reference means _____

- (A) That the recently executed instruction won't be executed soon
- (B) That the recently executed instruction is temporarily not referenced
- (C) That the recently executed instruction will be executed soon again
- (D) None of the mentioned

(A) (A)

(B) (B)

(C) (C)

Correct answer

(D) (D)

Quiz 3



UNIVERSITY
of York

Question 10

1 point



The algorithm to remove and place new contents into the cache is called _____

- (A) Replacement algorithm
- (B) Renewal algorithm
- (C) Updation
- (D) None of the mentioned

(A) (A)

Correct answer

(B) (B)

(C) (C)

(D) (D)

As the cache gets full, older contents of the cache are swapped out with newer contents. This decision is taken by the algorithm.

Quiz 3

Question 11

1 point



Typically, a split cache divides both the L1 and L2 caches into an instruction-only cache and a data-only cache.

True

False

Correct answer

ONLY L1 caches are divided into instruction and data caches. A single L2 cache should feed both L1 caches.

Quiz 3



UNIVERSITY
of York

1 point

...

Question 12

Assume a memory access to main memory on a cache "miss" takes 30 ns and a memory access to the cache on a cache "hit" takes 3 ns. If 80% of the processor's memory requests result in a cache "hit", what is the average memory access time?

- (A) 8.4 nS
- (B) 33 nS
- (C) 24.6 nS
- (B) 27.0 nS

(A) (A)

Correct answer

(B) (B)

(C) (C)

(D) (D)

For this question, you need to see that 80% of the time, the access will be 3 ns while the rest of the time (20%) the access time will be 30 ns. You should be able to see then that the average is going to be closer to 3 ns than 30 ns meaning the answer is a. You could also see this with the following equation:

$$(0.8 \times 3 \text{ ns}) + (0.2 \times 30 \text{ ns}) = 2.4 \text{ ns} + 6 \text{ ns} = 8.4 \text{ ns}$$

Quiz 3



UNIVERSITY
of York

Question 13

1 point

...

A page fault occurs when:

- ☐ A - The CPU accesses cache memory
- ☒ B - A page is not found in physical memory
- ☐ C - The TLB is full
- ☐ D - The page size is too small

Correct answer

A page fault happens when a referenced page is not in RAM and must be fetched from disk.

Quiz 3



UNIVERSITY
of York

Question 14

1 point

...

What does the Translation Lookaside Buffer (TLB) store?

- ☐ A - Data values
- ☐ B - Instructions
- ☒ C - Page table entries
- ☐ D - Cache lines

Correct answer

The TLB caches recent page table entries to speed up address translation.

Quiz 3



UNIVERSITY
of York

Question 15

1 point

...

Which page replacement algorithm replaces the page that has not been used for the longest time?

☐ A - FIFO

☒ B - LRU

Correct answer

☐ C - Optimal

☐ D - Random

Least Recently Used (LRU) removes the page least recently accessed.

Quiz 3



UNIVERSITY
of York

Question 16

1 point

...

Which page replacement algorithm is theoretical and used as a benchmark?

- ☐ A - FIFO
- ☐ B - LRU
- ☒ C - Optimal
- ☐ D - Clock

Correct answer

The Optimal algorithm replaces the page that will not be used for the longest time in the future.

Quiz 3



UNIVERSITY
of York

Question 17

1 point

...

What is the main goal of cache memory?

- ☐ A - Increase disk speed
- ☒ B - Reduce memory access latency
- ☐ C - Increase RAM size
- ☐ D - Replace registers

Correct answer

Cache stores frequently accessed data closer to the CPU to reduce access time.

Quiz 3



UNIVERSITY
of York

Question 18

1 point

...

Which mapping technique allows a block to be placed anywhere in cache?

- ☐ A - Direct mapping
- ☐ B - Set-associative
- ☒ C - Fully associative
- ☐ D - Indexed mapping

Correct answer

Fully associative mapping allows any memory block in any cache line.

Quiz 3



UNIVERSITY
of York

Question 19

1 point

...

Which mapping has the simplest hardware implementation?

☐ A - Fully associative

☐ B - Set-associative

☒ C - Direct mapped

Correct answer

☐ D - Hybrid

Direct mapping uses a fixed location, making it simple and fast.

Quiz 3



UNIVERSITY
of York

Question 20

1 point

...

Which block replacement policy is commonly used in set-associative caches?

☐ A - FIFO

☒ B - LRU

Correct answer

☐ C - Optimal

☐ D - MRU

LRU is widely used for its good performance trade-off.

Quiz 3



UNIVERSITY
of York

Question 21

1 point

...

What is a dirty bit used for?

- ☐ A - Detect cache hits
- ☒ B - Indicate modified data in cache
- ☐ C - Track page faults
- ☐ D - Control TLB

Correct answer

It marks whether a cache block has been modified.

Quiz 3



UNIVERSITY
of York

Question 22

1 point

...

Increasing cache size generally:

- ☒ A - Increases hit rate
- ☐ B - Decreases hit rate
- ☐ C - Has no effect
- ☐ D - Eliminates misses

Correct answer

Larger caches store more data, improving hit probability.

Quiz 3



UNIVERSITY
of York

Question 23

1 point

...

Which technique reduces conflict misses?

- ☐ A - Direct mapping
- ☒ B - Increasing associativity
- ☐ C - Decreasing block size
- ☐ D - Removing cache

Correct answer

Higher associativity allows more flexible placement.

Quiz 3



UNIVERSITY
of York

Question 24

1 point

...

Which optimisation preloads data before it is needed?

☐ A - Write-back

☒ B - Prefetching

Correct answer

☐ C - LRU

☐ D - Paging

Prefetching anticipates future accesses to reduce miss penalties.

Discussion Forum

Discussion Forum - Key Themes for Discussion



1. Pipeline Efficiency and Optimisation Techniques

- Focus on achieving ideal CPI (Cycles Per Instruction) in pipelined architectures.
- Explore compiler techniques like instruction scheduling, loop unrolling, and instruction reordering to expose Instruction-Level Parallelism (ILP) and reduce pipeline stalls.

2. Dynamic Scheduling and Hazard Resolution

- Discuss how dynamic scheduling methods, such as scoreboarding and Tomasulo's algorithm, help manage hazards (RAW, WAR, WAW).
- Understand the role of reservation stations and load/store buffers in enabling out-of-order execution and improving instruction throughput.

Discussion Forum - Key Themes for Discussion



3. Balancing Performance Gains and Practical Limitations

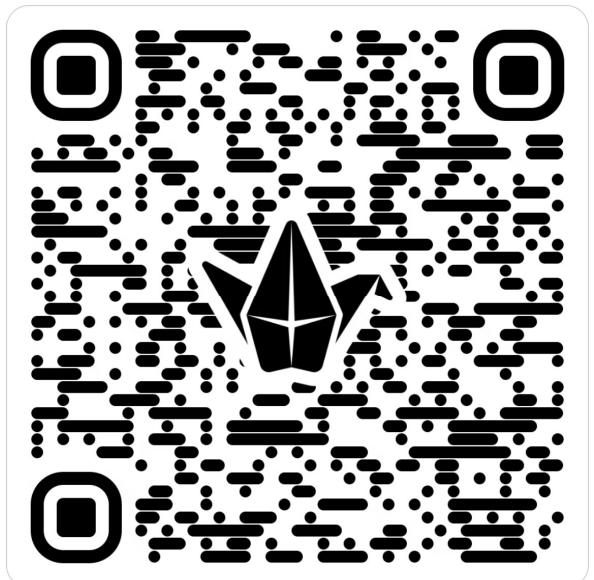
- Examine the trade-offs and limitations of optimisation techniques (e.g., loop unrolling causing register pressure).
- Consider hardware constraints and design complexities that limit performance improvements despite advanced techniques.

Discussion Forum -



Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

1. What is the ideal pipeline CPI?
2. What are compiler techniques, and why are they important in advanced computer architecture?
3. What is the role of instruction scheduling in compiler optimisation.
4. Explain loop unrolling and its benefits in optimising code performance.
5. What are the effects that limit the gains from loop unrolling?

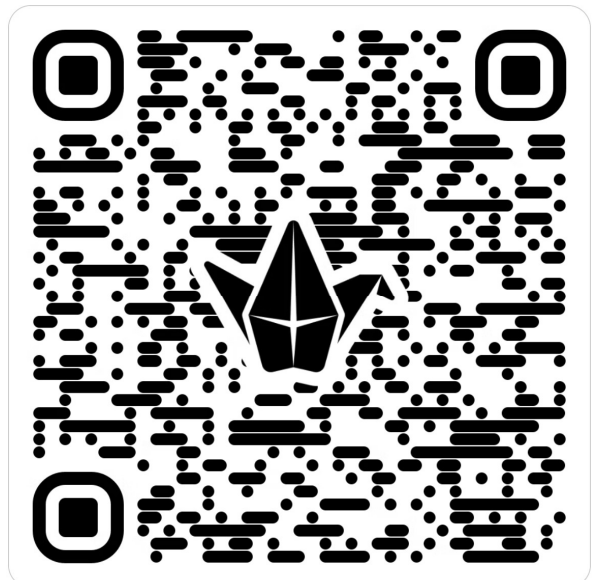


Discussion Forum - Pipeline & Compilers



Pick one or more of the following questions, discuss with your small group and add comments to the Padlet discussion board:

6. How does instruction reordering help alleviate hazards in the RISC pipeline?
7. What is score-boarding? How is it being used for instructions to execute out of order?
8. Explain the function of a “reservation station” in Tomasulo's algorithm (without speculation)
9. What are the benefits in using reservation stations rather than a centralised register file?
10. What are the main functions of the load buffers and store buffers using Tomasulo's algorithm?



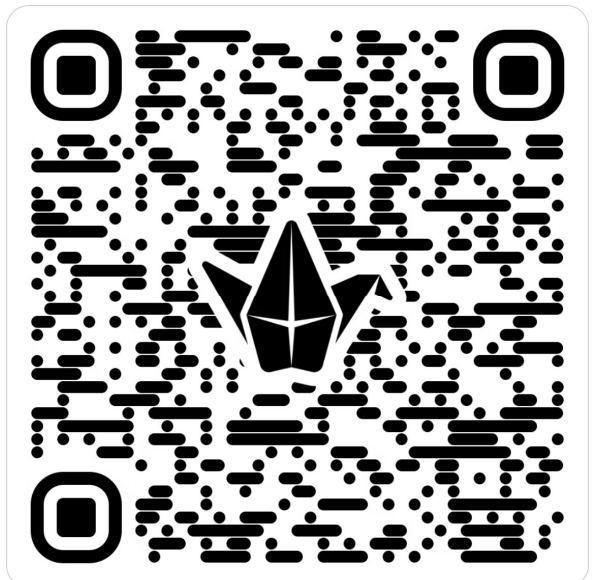
Discussion Forum - Approach



Small Group Discussions

- Form a small group with someone nearby, or feel free to move around and join your friends
- Pick one or more applications
- 15 mins group discussion
- Add comments/summary as a group to the Padlet discussion board

<https://padlet.com/evanismeuk/week-8-4cl4f8zh612u92oq>



Discussion Forum - Compiler Techniques



1. What is the ideal pipeline CPI?

The value of the CPI (cycles per instruction) for a pipelined processor is the sum of the base CPI and all contributions from stalls:

Pipeline CPI Ideal pipeline = CPI + Structural stalls + Data hazard stalls + Control stalls

The ideal pipeline CPI is a measure of the maximum performance attainable by the implementation. By reducing each of the terms of the right-hand side, we decrease the overall pipeline.

Discussion Forum - Compiler Techniques



2. What are compiler techniques, and why are they important in advanced computer architecture?

Compiler techniques are methods, algorithms, and strategies used by compilers to translate high-level programming languages into executable machine code optimized for a target computer architecture. These techniques play a crucial role in advanced computer architecture for several reasons:

Compiler techniques **optimize code** to improve program performance on the target architecture. By analyzing program structure, data dependencies, and hardware characteristics, compilers can generate code that maximizes instruction-level parallelism, minimizes memory access latency, and optimally utilizes hardware resources like caches and pipelines.

Advanced computer architectures often feature complex hardware structures with multiple processing units, caches, and specialized functional units.

Compiler techniques ensure **efficient utilization of these resources** by scheduling instructions, managing memory accesses, and minimizing resource contention, thus enhancing overall system efficiency.

Discussion Forum - Compiler Techniques



2. What are compiler techniques, and why are they important in advanced computer architecture?

With the growing emphasis on **energy-efficient computing**, compiler techniques are vital for minimizing power consumption in modern processors. Techniques like loop unrolling, instruction scheduling, and register allocation help reduce unnecessary computations, minimize memory accesses, and optimize code paths, leading to lower energy consumption during program execution.

Compiler techniques enable the generation of **portable code** that can run efficiently across different architectures and platforms. By abstracting hardware details and optimizing code at the high-level language level, compilers facilitate software development and deployment on diverse computing environments without sacrificing performance or functionality.

Discussion Forum - Compiler Techniques



2. What are compiler techniques, and why are they important in advanced computer architecture?

Advanced computer architectures often have limited memory resources, especially in embedded systems and mobile devices. Compiler techniques such as instruction compression, dead code elimination, and function inlining reduce code size, conserve memory space, and improve cache utilization, thereby **enhancing system scalability and performance**.

Discussion Forum - Compiler Techniques



3. Discuss the role of instruction scheduling in compiler optimization.

Instruction scheduling is a crucial compiler optimization technique aimed at rearranging program instructions to improve performance by maximizing instruction-level parallelism and minimizing pipeline stalls. Here's how instruction scheduling contributes to compiler optimization:

In modern processors, instruction execution is often pipelined, where different stages of instruction execution occur concurrently. However, dependencies between instructions can lead to pipeline stalls, where the processor must wait for an instruction to complete before executing the next one. Instruction scheduling aims to reorder instructions **to reduce or eliminate these stalls**, thereby improving pipeline efficiency and overall performance.

Instruction scheduling **exploits different types of parallelism** available in the program, including instruction-level parallelism (ILP) and data-level parallelism (DLP). By reordering instructions to execute independent operations concurrently, instruction scheduling increases the number of instructions in flight and maximizes hardware utilization, leading to improved performance.

Discussion Forum - Compiler Techniques



3. Discuss the role of instruction scheduling in compiler optimization.

Dependencies between instructions, such as data dependencies (read-after-write, write-after-read, write-after-write), limit the degree of parallelism that can be exploited. Instruction scheduling **analyses data dependencies and reorders instructions to minimize their impact**. Techniques such as register renaming, data forwarding, and speculative execution are used to mitigate dependencies and facilitate instruction reordering.

Instruction scheduling optimizes resource utilization within the processor by ensuring that hardware resources are efficiently utilized. By scheduling instructions to balance resource usage and avoid resource contention, instruction scheduling **minimises bottlenecks and improves overall system throughput**.

Discussion Forum - Compiler Techniques



3. Discuss the role of instruction scheduling in compiler optimization.

Modern processors have pipelines with **varying latencies** for different types of instructions and operations. Instruction scheduling takes into account these pipeline latencies and schedules instructions to **minimise the overall execution time**. By scheduling instructions with shorter latencies first and interleaving long-latency instructions with independent operations, instruction scheduling reduces program execution time and improves performance.

Instruction scheduling techniques are tailored to specific processor architectures and pipeline structures. Different processors have different pipeline lengths, latencies, and dependencies, requiring customized instruction scheduling algorithms. Compiler developers design scheduling algorithms that are optimized for target architectures to maximize performance while adhering to architectural constraints.

Discussion Forum - Compiler Techniques



4. Explain loop unrolling and its benefits in optimising code performance.

Loop unrolling is a compiler optimisation technique used to improve the performance of loops by reducing loop overhead and increasing instruction-level parallelism. It involves replicating loop bodies multiple times to reduce the number of loop iterations, thereby reducing the overhead associated with loop control and improving the efficiency of instruction execution. Here's how loop unrolling works and its benefits in optimizing code performance:

Basic Concept of Loop Unrolling:

In loop unrolling, the compiler replicates the body of a loop multiple times within the source code, effectively increasing the size of the loop body and reducing the number of loop iterations required to achieve the same computation.

Instead of executing the loop with each iteration, the unrolled loop executes multiple iterations of the loop body within each iteration of the unrolled loop, resulting in fewer loop control instructions and reduced loop overhead.

Discussion Forum - Compiler Techniques



4. Explain loop unrolling and its benefits in optimizing code performance.

Benefits of Loop Unrolling:

Loop unrolling reduces the overhead associated with loop control instructions, such as loop counters, loop termination conditions, and loop branch instructions. This **reduction in overhead** improves the overall efficiency of the loop and enhances performance.

Loop unrolling exposes additional opportunities for instruction-level parallelism by **allowing multiple loop iterations to execute concurrently**. This enables the processor to execute multiple loop iterations in parallel, effectively overlapping instruction execution and reducing execution latency.

Loop unrolling can **improve memory access patterns** by increasing the locality of reference within the loop body. With fewer loop iterations, the unrolled loop accesses contiguous memory locations more frequently, reducing cache misses and improving data cache utilization.

Discussion Forum - Compiler Techniques



4. Explain loop unrolling and its benefits in optimizing code performance.

Further considerations:

While loop unrolling can improve performance, excessive unrolling **may lead to code bloat and increased code size**, potentially negating the benefits of optimization.

The effectiveness of loop unrolling depends on the characteristics of the loop, such as loop trip count, loop body complexity, and memory access patterns. Not all loops are suitable candidates for unrolling, and **careful analysis is required to determine the optimal unrolling factor**.

Discussion Forum - Compiler Techniques



5. What are the effects that limit the gains from loop unrolling?

While loop unrolling can bring performance benefits in terms of improved instruction scheduling and increased instruction-level parallelism, there are certain factors and effects that may limit the gains from loop unrolling. Some of these limitations include:

Loop unrolling can lead to an increase in code size because each iteration is expanded into multiple copies of the loop body. This **may result in a larger executable**, potentially impacting instruction cache efficiency and overall memory usage.

Unrolling a loop **increases the number of variables and expressions** that need to be kept in registers. If the loop body is already register-intensive or if the target architecture has a limited number of registers, loop unrolling may lead to **increased register pressure**, causing spills to memory and degrading performance.

Discussion Forum - Compiler Techniques



5. What are the effects that limit the gains from loop unrolling?

For loops with a small number of iterations, the benefits of loop unrolling may be limited. If the loop runs for only a few iterations, the overhead of unrolling may not be justified, and the gains from improved scheduling may not be significant.

Loop unrolling often involves fewer conditional branches within the loop body, but it **may introduce additional conditional branches** to handle the remaining iterations that are not part of the unrolled loop. This can **lead to branch mispredictions** and limit the effectiveness of loop unrolling.

Some loops **may have dependencies** or control flow characteristics that make them less amenable to unrolling. For example, loops with irregular or data-dependent exit conditions may not be suitable candidates for effective loop unrolling.

Discussion Forum - Compiler Techniques



5. What are the effects that limit the gains from loop unrolling?

The effectiveness of loop unrolling is dependent on the compiler's ability to analyze and optimize the loop structure. Some compilers may have limitations in identifying opportunities for loop unrolling or may choose not to unroll loops based on heuristics.

Loop unrolling can affect the memory access pattern and cache behaviour. While it may improve cache locality in some cases, it can also introduce new challenges, such as **increased pressure on the instruction cache** or altered patterns of data accesses.

In some cases, the available ILP may already be saturated, and further loop unrolling may not result in additional parallelism. The gains from increased ILP may be limited if the processor is already fully utilizing available resources.

Developers must carefully analyse the specific code, target architecture, and performance goals when considering loop unrolling. Experimentation, profiling, and collaboration with the compiler may be necessary to find the optimal balance between loop unrolling and other optimization strategies.

Discussion Forum - Compiler Techniques



6. How does instruction reordering help alleviate hazards in the RISC pipeline?

Instruction reordering is a technique used to alleviate hazards in the RISC pipeline by rearranging the order of instructions to optimize instruction execution and minimize pipeline stalls. Here's how instruction reordering helps alleviate hazards in the RISC pipeline:

Instruction reordering allows the compiler or hardware to **schedule instructions** in such a way that **data dependencies are minimised**. By rearranging instructions, the pipeline can execute independent instructions concurrently, reducing the likelihood of data hazards and associated pipeline stalls.

When hazards occur, such as data hazards or control hazards, pipeline stalls can result in idle pipeline stages, or "pipeline bubbles." Instruction reordering aims to **fill these bubbles by scheduling independent instructions to execute during hazard delays**, maximizing pipeline throughput and efficiency.

Discussion Forum - Compiler Techniques



6. How does instruction reordering help alleviate hazards in the RISC pipeline?

Instruction reordering **enables speculative execution of instructions** along predicted execution paths. By reordering instructions based on branch prediction outcomes, the pipeline can continue executing instructions along the predicted path, even before the branch outcome is known. This reduces the impact of control hazards and minimizes pipeline stalls.

Reordering instructions can help **optimise resource utilisation** within the pipeline. By scheduling instructions to minimize structural hazards, where multiple instructions contend for the same hardware resource simultaneously, instruction reordering improves overall pipeline efficiency and performance.

Discussion Forum - Compiler Techniques



6. How does instruction reordering help alleviate hazards in the RISC pipeline?

Instruction reordering is often performed by compilers during code optimization phases. Compilers analyze the data and control dependencies within the code and reorder instructions to minimize hazards and maximize parallelism. Compiler-generated code sequences are designed to exploit pipeline features efficiently and reduce the likelihood of stalls.

In some RISC processors, instruction reordering is performed dynamically by hardware using out-of-order execution techniques. Instructions are dispatched and executed out of program order based on available resources and data dependencies. This allows the processor to maintain high throughput and performance even in the presence of hazards.

Discussion Forum - Dynamic Scheduling



7. What is score-boarding? How is it being used for instructions to execute out of order?

Scoreboarding is a hardware-based technique used in computer processors to implement out-of-order execution. It is designed to **manage dependencies and hazards**, allowing instructions to be executed as soon as their operands are available, rather than strictly in program order. Scoreboarding is a form of dynamic scheduling that helps maximize instruction-level parallelism (ILP) and improve overall processor efficiency. Here's how it works and how it enables out-of-order execution:

In a scoreboarding system, **each functional unit (e.g., an arithmetic logic unit or ALU) has associated reservation stations**. These reservation stations hold instructions that are waiting for their operands and are ready to execute when the operands become available.

Each functional unit has a **status flag** indicating whether it is busy or available. If it is busy, the reservation station is holding an instruction; if it is available, the reservation station is free for the next instruction.

Discussion Forum - Dynamic Scheduling



7. What is score-boarding? How is it being used for instructions to execute out of order?

Issue (Issue Queue): Instructions are fetched and decoded in order. When an instruction is ready to execute, it is issued to an available reservation station associated with the required functional unit. The issue queue is responsible for scheduling instructions to reservation stations.

Operand Read (Read Operands): Before an instruction is executed, the operands are read from the register file or the result of a previous instruction in the pipeline. If the required operands are not available (e.g., due to a data hazard), the instruction waits in the reservation station until the operands are ready.

Execution: Once the operands are available, the instruction can be executed. The functional unit performs the operation specified by the instruction.

Write Result (Write Result): After execution, the result is written back to the register file, and the associated reservation station is marked as available.

Other instructions that are waiting for this result can now proceed.

Discussion Forum - Dynamic Scheduling



7. What is score-boarding? How is it being used for instructions to execute out of order?

Commit (Commit Unit): Instructions are committed in program order. The commit unit ensures that the instructions are retired in the correct order, preserving the program's logical correctness.

By using scoreboarding, instructions that do not have dependencies can be executed out of order as soon as their operands are available. The hardware dynamically schedules instructions to functional units based on the availability of resources, improving parallelism and overall execution efficiency.

Scoreboarding helps overcome hazards, such as data hazards and resource contention, by allowing instructions to execute independently as soon as their prerequisites are satisfied.

While scoreboarding is an effective technique for out-of-order execution, more modern processors often use more advanced techniques like Tomasulo's algorithm or reservation station-based schemes with additional optimizations to further enhance performance and flexibility in handling dependencies.

Discussion Forum - Dynamic Scheduling



8. Explain the function of a “reservation station” in Tomasulo's algorithm (without speculation)

Whenever an instruction is issued, all information necessary to executing the instruction is stored in its reservation station. The reservation station's ID also identifies what value should be used by subsequent instructions depending on the instruction's result.

Discussion Forum - Dynamic Scheduling



9. What are the benefits in using reservation stations rather than a centralized register file?

The use of reservation stations, rather than a centralized register file, leads to two other important properties. First, **hazard detection and execution control are distributed**: The information held in the reservation stations at each functional unit determines when an instruction can begin execution at that unit. Second, **results are passed directly to functional units** from the reservation stations where they are buffered, rather than going through the registers. This bypassing is done with a common result bus that allows all units waiting for an operand to be loaded simultaneously. In pipelines with multiple execution units and issuing multiple instructions per clock, more than one result bus will be needed.

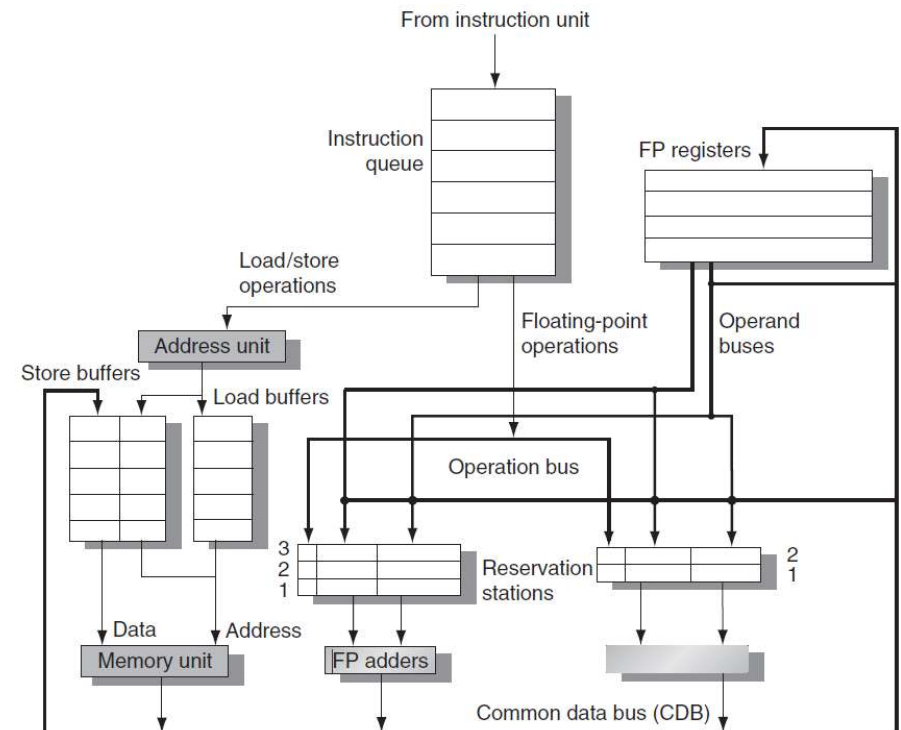
Discussion Forum - Dynamic Scheduling

10. What are the main functions of the load buffers and store buffers using Tomasulo's algorithm?

The load buffers and store buffers hold data or addresses coming from and going to memory and behave almost exactly like reservation stations.

Load buffers have three functions: (1) hold the components of the effective address until it is computed, (2) track outstanding loads that are waiting on the memory, and (3) hold the results of completed loads that are waiting for the CDB.

Similarly, store buffers have three functions: (1) hold the components of the effective address until it is computed, (2) hold the destination memory addresses of outstanding stores that are waiting for the data value to store, and (3) hold the address and value to store until the memory unit is available.



Closing Remarks and Announcement

- **Next Week (Week 10):**

Lecture: In Week 10, our discussion will be focused on some cache memory optimization techniques in Lecture 17 and the 4th Tutorial Lecture on Caches.

Lab: In Week 10 there will be no algorithm coding. However we will use the pipeline toolset to explore some tradeoffs relating to power consumption of the CPU and the effect of optimisation.

Exercises/Quizzes:

Exercise 3 will be discussed in the Lecture Review session on 30/04/2026.

Thanks

See you next week!